

옆집 손주

노인 복지 및 법률 상담 챗봇



SKN28-3rd-1Team

김지효 송윤경 양도영 이원빈 전하영

팀원 소개



이원빈
역할 : 팀장



전하영
역할 : 백엔드



김지효
역할 : RAG



양도영
역할 : 기획, 문서



송윤경
역할 : 프론트엔드

CONTENTS

01) 프로젝트 목표 및 방향성

02) 프로젝트 구현 기술

03) 벤치마킹 및 검증

04) 프로젝트 데모





SECTION 01

프로젝트 목표 및 방향성

01

프로젝트 배경, 주제, 핵심 목표

프로젝트 진행 배경

흩어진 정보 확인과 최신 정보 갱신의 필요성



프로젝트 주제 : 노인과 고령층이 법률, 기초연금, 고령자 고용, 근로 관련 정보를 자연어로 질문하고, 실제 공공 문서와 법령을 근거로 답변을 받을 수 있도록 만드는 RAG 기반 상담 서비스 챗봇 제작

• 문제 상황

01 흩어진 정보

노인복지, 기초연금, 고령자 고용, 근로 관련 정보가 여러 기관에 나뉘어 있어 한 번에 찾기 어려운 문제

02 용어 문제

어려운 용어로 인해 법령과 행정 문서는 일반 사용자, 특히 고령층의 경우 바로 이해하기 어려운 문제

03 최신 정보 갱신 문제

연금 기준, 지원 금액, 고용 기준은 바뀔 수 있는 영역이기 때문에 계속해서 갱신된 최신 문서 확인이 필요

04 잘못된 정보 제공 문제

복지, 법률 정보는 잘못 안내되면 실제 불이익으로 이어질 수 있는 위험 존재

프로젝트 핵심 목표

문서 기반 검색 챗봇과 RAG 활용



- 프로젝트 목표

목표	설명
 문서 기반 검색	노인·고령층 관련 법령과 행정 안내문을 검색 가능한 형태로 정리합니다.
 Agentic RAG 답변	Main Agent가 질문을 판단하고 필요한 경우 RAG MCP Tool을 호출합니다.
 근거 중심 응답	RAG 검색 결과와 출처를 바탕으로 답변합니다.
 상담형 화면	Streamlit 프로토타입으로 질문 입력부터 답변 확인까지 검증합니다.
 추적 가능성	LangSmith로 LLM 호출과 tool calling 과정을 확인합니다.

- RAG 필요성

일반 LLM은 학습된 지식만으로 답변하기 때문에 최신 법령, 기초연금 신청 기준, 고령자 고용 기준을 정확히 보장하기 어렵지만, 이 프로젝트를 실제 문서를 먼저 찾고, 그 문서를 바탕으로 답변하는 구조 사용

주요 사용자

노인 및 고령층 뿐만 아니라 복지 및 법률 관련 실무자들을 위한 챗봇



• 주요 예상 챗봇 사용자



노인·고령층
당사자와 가족

기초연금·신청 조건·권리 보호 절차



복지사 및
상담 실무자

상담 중 빠르게 확인할 법령과 공공
문서 근거



고령자 고용
관련 실무자

고령자 고용·연령차별·근로 기준 법령

• 예상 질문

예상 질문 예시

- 65세 이상 노인이 받을 수 있는 혜택은 어떤 것이 있나요?
- 기초연금 신청 방법과 준비 해야할 서류들은 어떤 것이 있나요?
- 나이 때문에 채용에서 차별받으면 어떻게 대처해야 할까요?
- 퇴직금을 못 받았을 때 확인할 방법은 어떤 것이 있을까요?



SECTION 02

프로젝트 구현 기술

02

FRONTEND, BACKEND, RAG

검수 가능한 GraphRAG 서비스



법령/조례 도메인은 의미 유사도 뿐 아니라 조항 간 관계, 예외, 근거 검수 필요

- 일반 vector RAG와의 비교

일반 vector RAG

Chunk

문서를 조각으로 나눔



Embedding

벡터화 후 top-k 검색



Answer

근거/관계 검수는 약함

유사 문장은 찾지만 조문 간 관계와 예외의 타당성을 검수하기 어려움

GraphRAG

Chunk + Embedding

의미 단위 chunk와
vector 저장



Graph Traverse

문서/조항/후보
관계 탐색



Candidate

edge가 아닌 검수
artifact



Human Review

승인 후보만 edge 확정

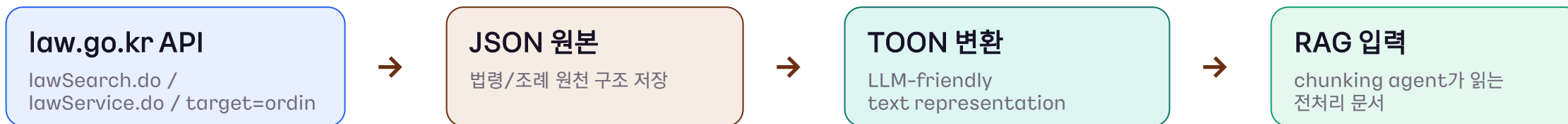
LLM은 제안하고, 사용자가 검수하며, DB에는 검증된 관계만 반영

토큰 절감

원본 API 구조를 보존하되 LLM 입력 비용을 줄이기 위해 JSON 문법 토큰 제거



- 토큰 절감 과정 요약



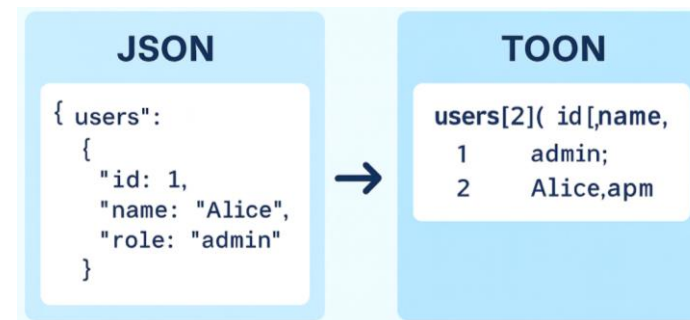
Token compression proof

320,991

원본 JSON tokens

189,997

TOON tokens



절감 토큰

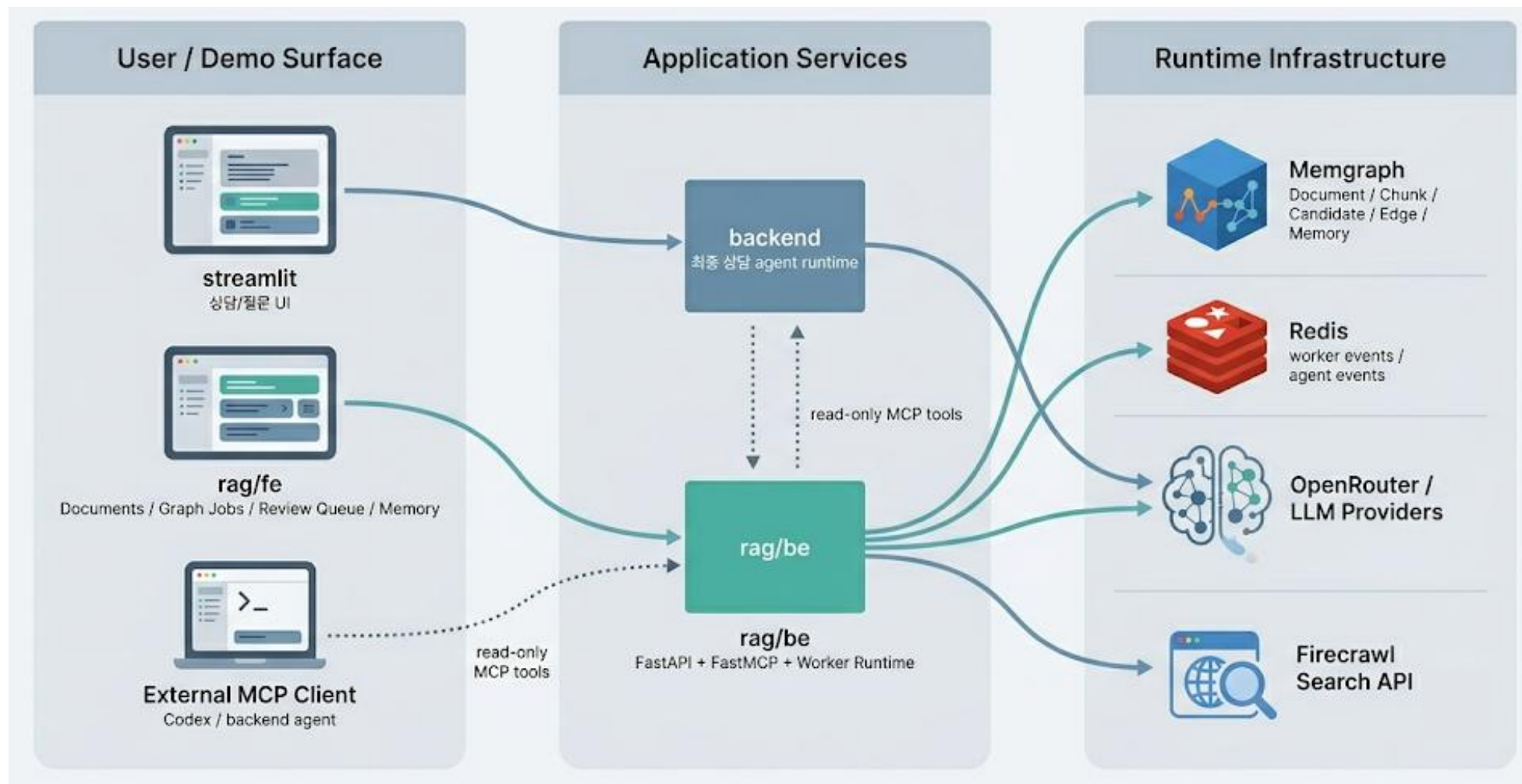
130,994

전체 합계 기준 절감률 40.81%. 문서별
평균이 아니라 전체 token sum 기준

근거: rag/code_reference 수집/전처리 코드, rag/RAG_PREPROCESSED_DATA/README.md

최종 서비스 구조

Frontend, LLM Runtime, GraphRAG Knowledge Layer로 분리



각 영역 역할 구조



최종 사용자 응답 runtime과 knowledge graph 구축 runtime 분리

- Frontend / Backend / RAG 분리

1 Streaming Frontend

사용자 질문 입력, token delta 렌더링, tool status/citation 표시

Presentation layer

2 Main Backend

session/context 구성, prompt policy, LLM 호출, MCP tool routing

Answer generation runtime

3 GraphRAG System

문서 업로드, graph build, review queue, memory, read-only MCP 제공

Knowledge construction layer

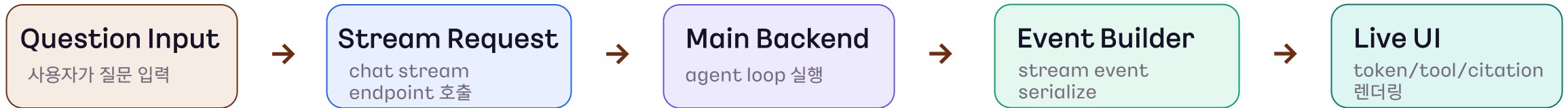
✓ RAG system 은 Knowledge Layer, Main Backend는 answer generation runtime

Frontend 실행 방식

RAG 내부를 직접 실행하지 않고 backend stream을 UX로 변환



- 답변 Flow



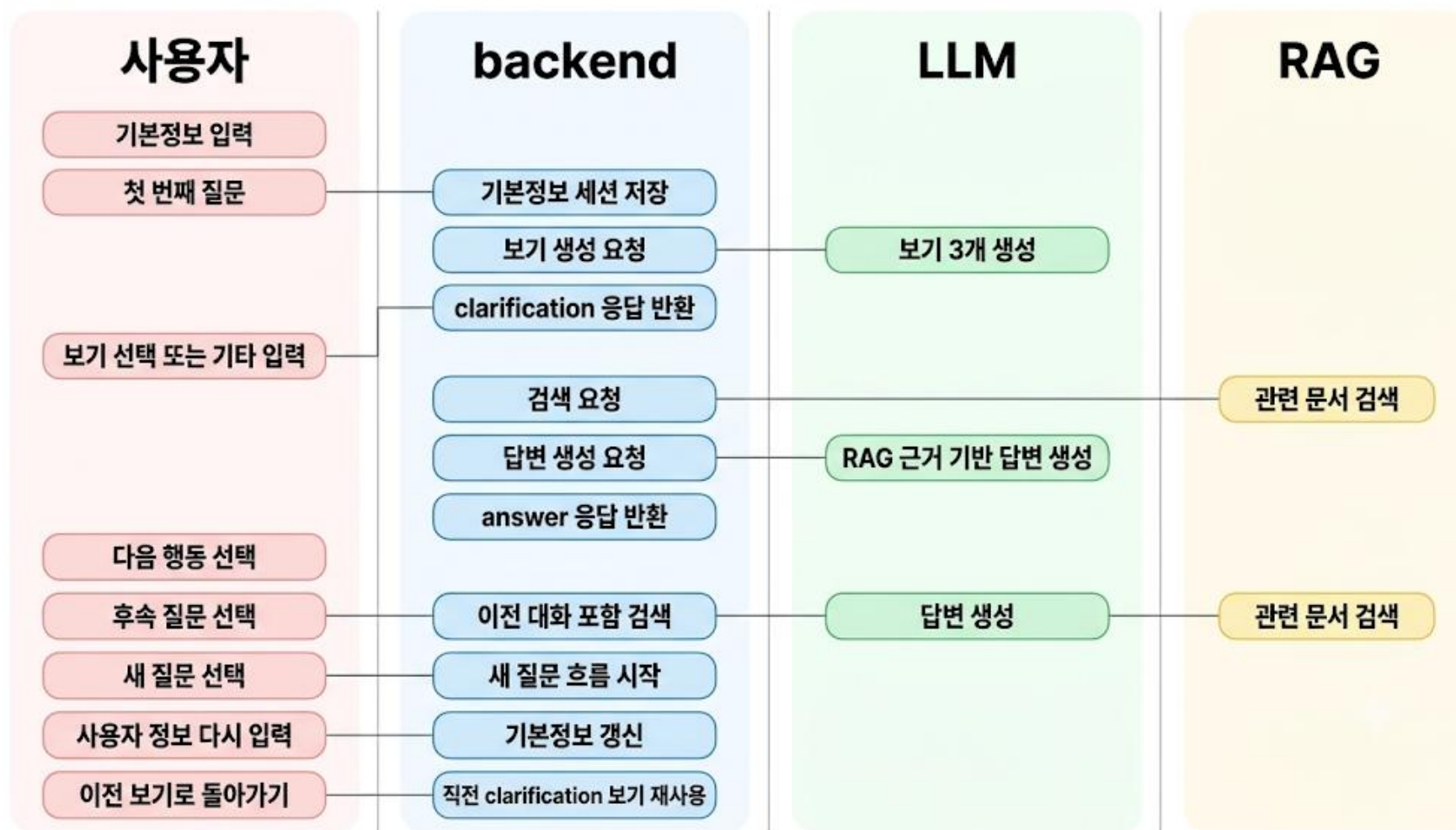
- Token, Tool status, Evidence event 순차적 제공



✓ Frontend slide에서는 RAG 구조를 설명하지 않고, stream lifecycle과 사용자 경험만 설명

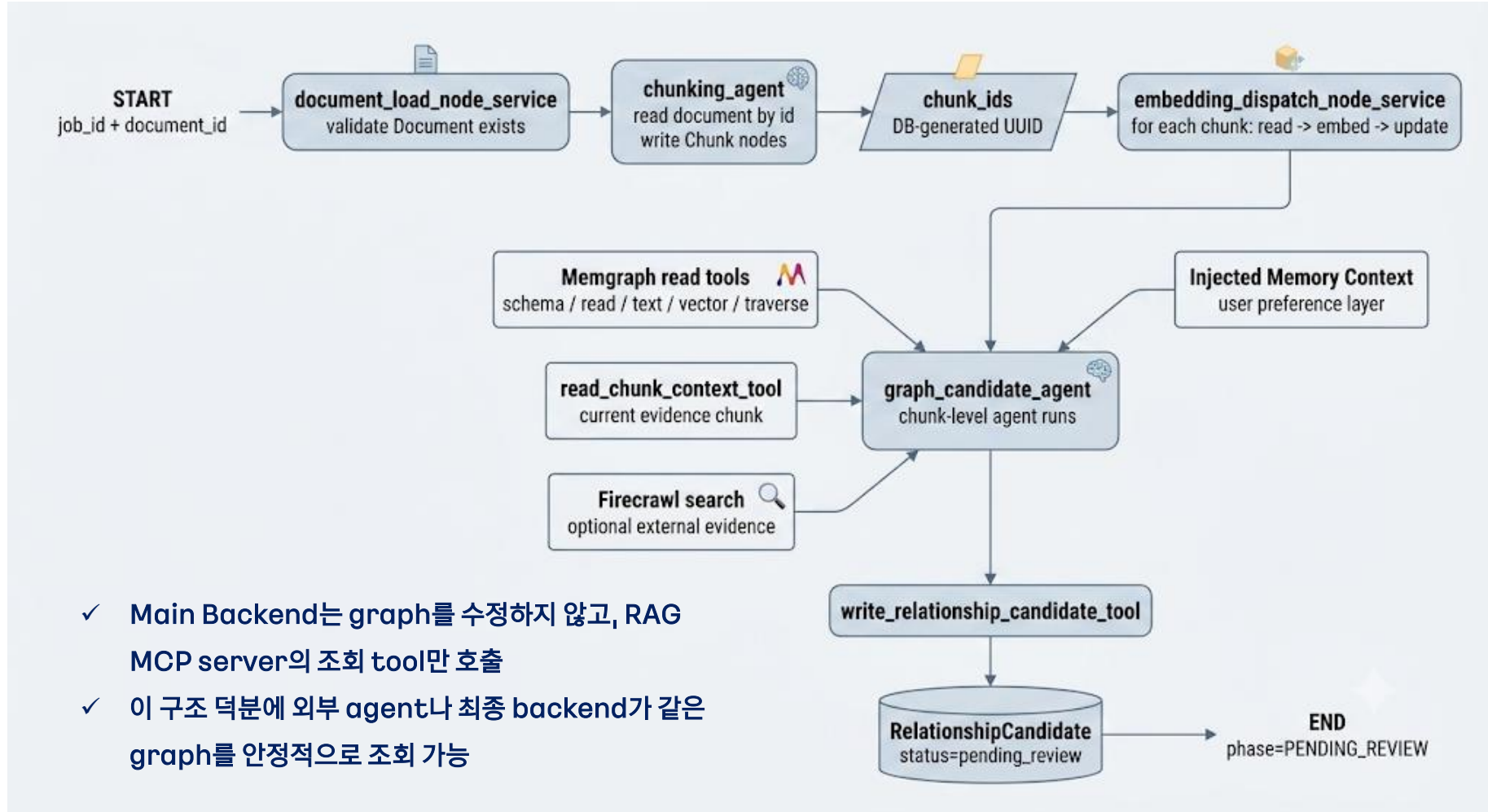
최종 답변 생성

최종 답변 생성은 backend가 담당하고 RAG는 MCP tool로 호출



MCP 역할

Answer runtime과 GraphRAG system 사이의 read-only boundary

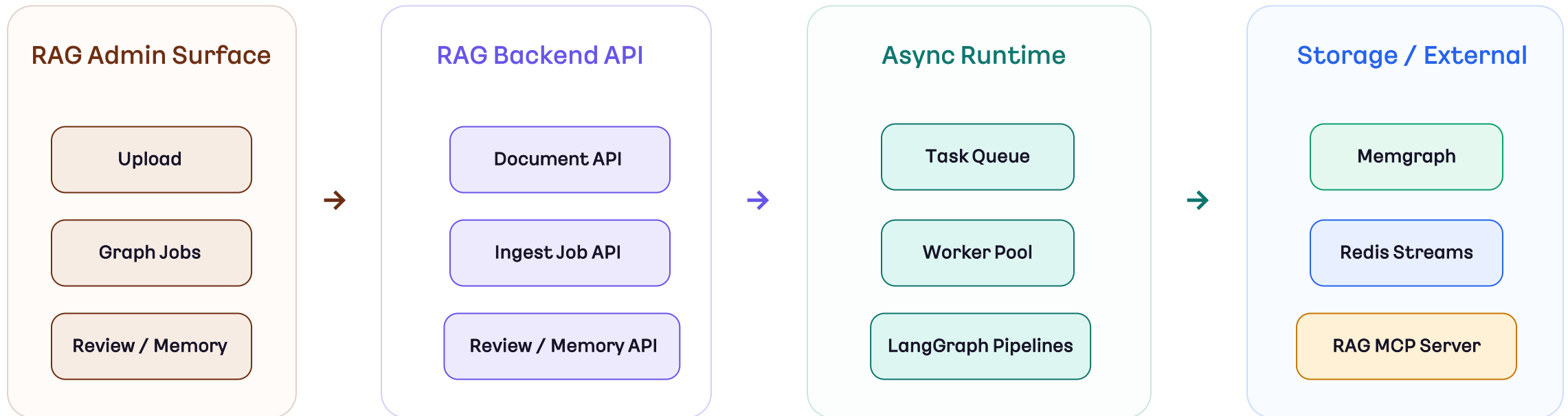


RAG system



Ingest, worker, review, memory, MCP를 포함한 graph 구축 시스템

- 최종 답변을 직접 생성하기보다, 검수 가능한 knowledge graph를 만들고 제공하는 구조



- ✓ RAG FE는 운영 UI, RAG BE는 API와 worker, Memgraph는 graph truth, Redis는 observability, MCP는 read-only 외부 조회 boundary

문서 업로드

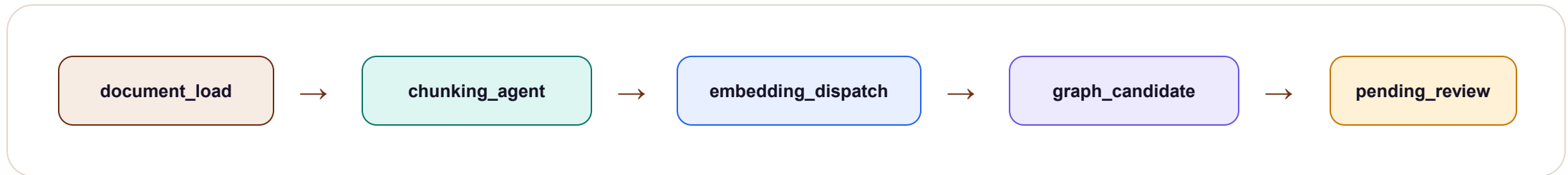


Job/queue/worker로 비동기 처리 후 graph construction 실행

- 문서 업로드 다이어그램



- Worker는 document load부터 candidate generation까지 진행하고 pending review artifact 제작



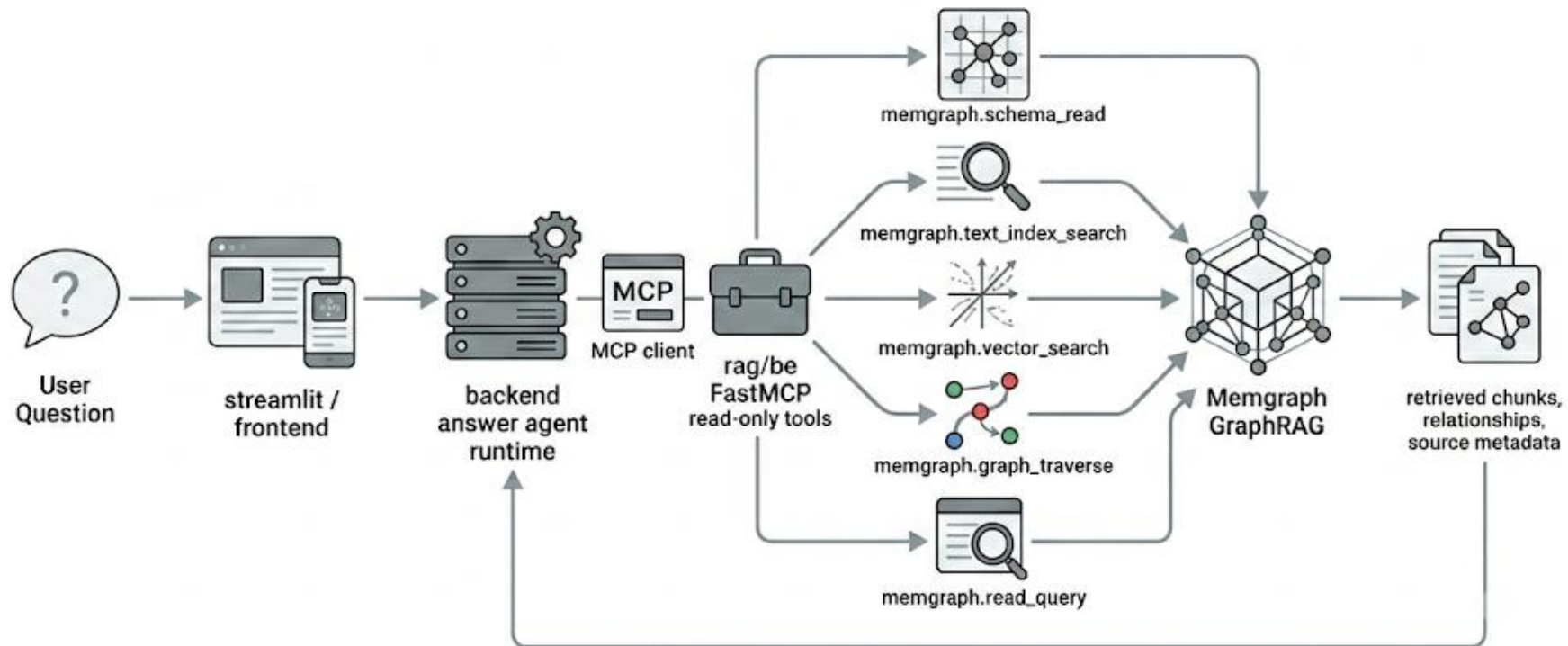
- Shared state에는 `job\_id`, `document\_id`, `chunk\_ids`만 실고 raw document는 DB/query boundary에 위치

Candidate First, Edge Later

LLM이 만든 관계는 바로 graph edge가 아니라 사람이 승인하는 Relationship Candidate



- 전체 다이어그램



➤ 검수 가능한 자동화: LLM output은 제안이고 DB graph truth는 review 이후에만 바뀐다



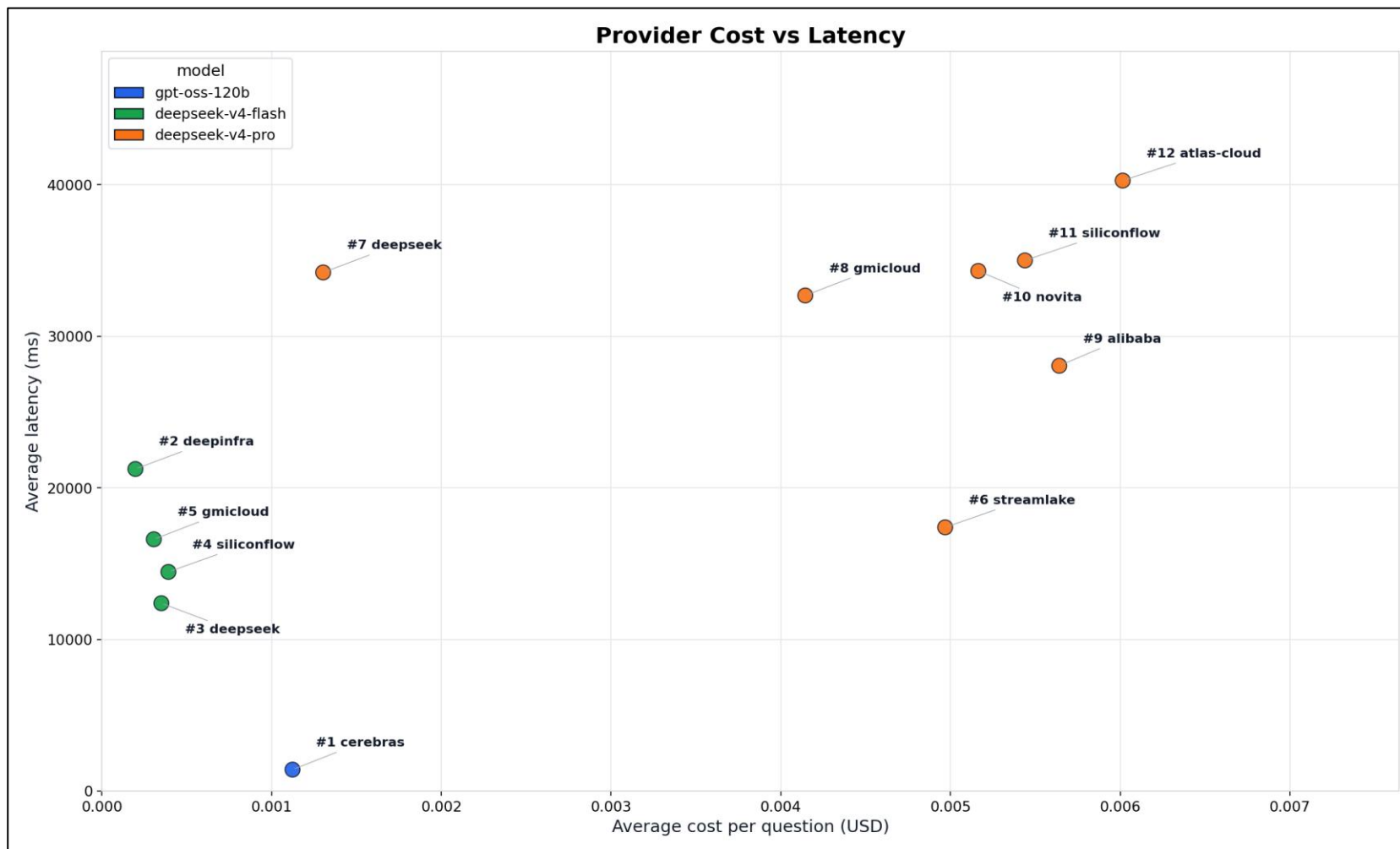
SECTION 03

벤치마킹 및 검증

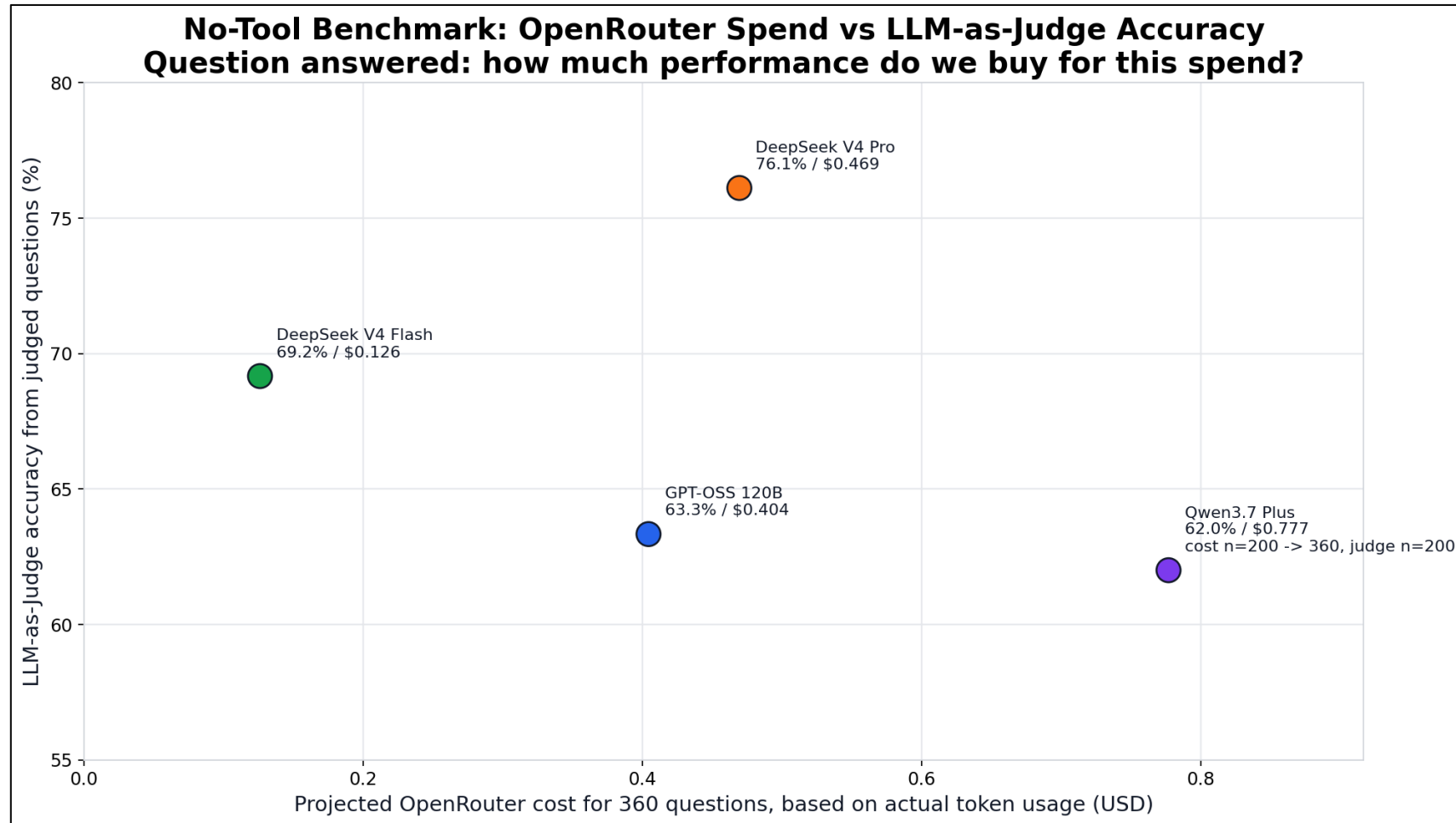
FRONTEND, BACKEND, RAG

03

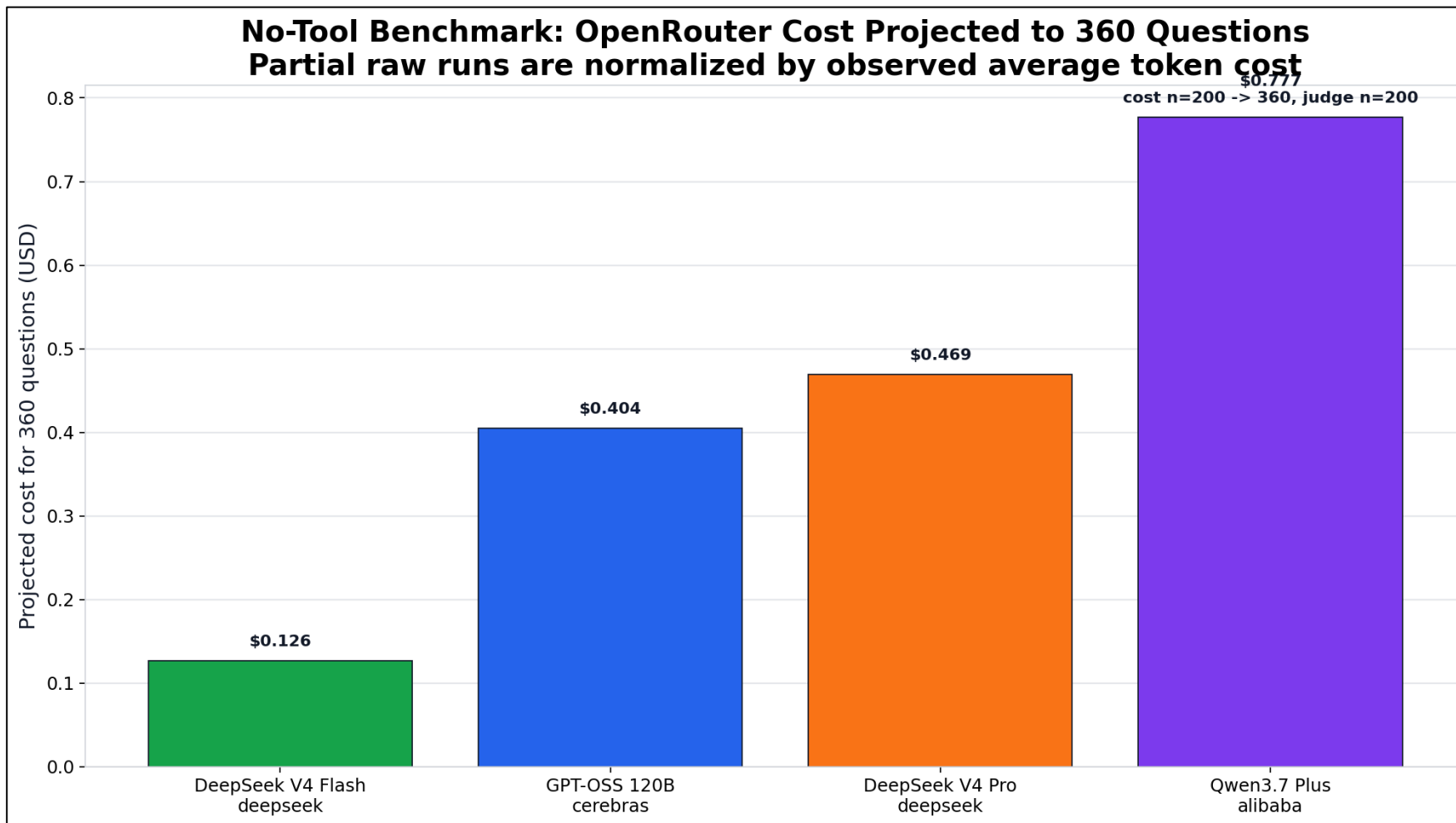
벤치마크 (1)



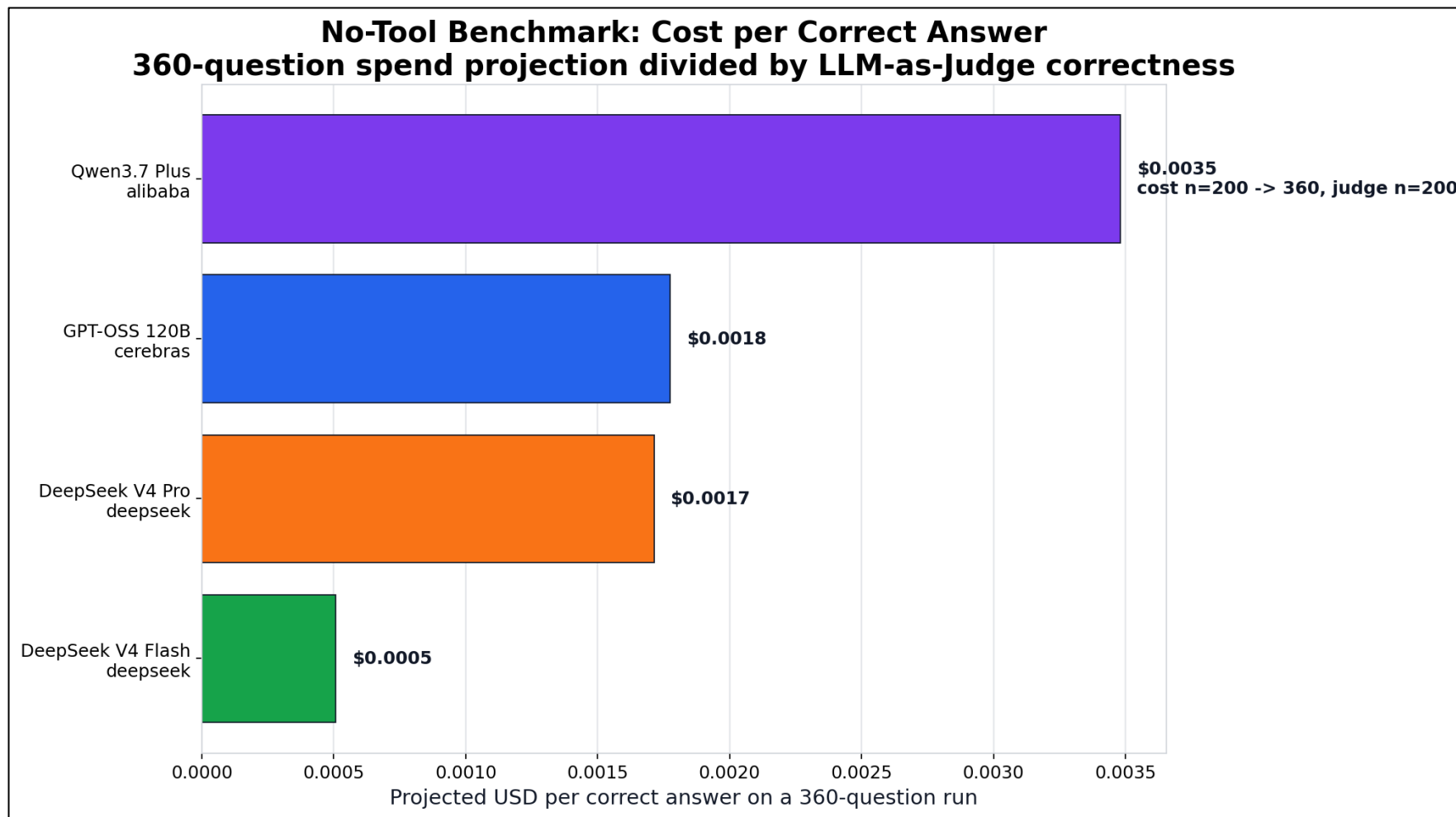
벤치마크 (2)



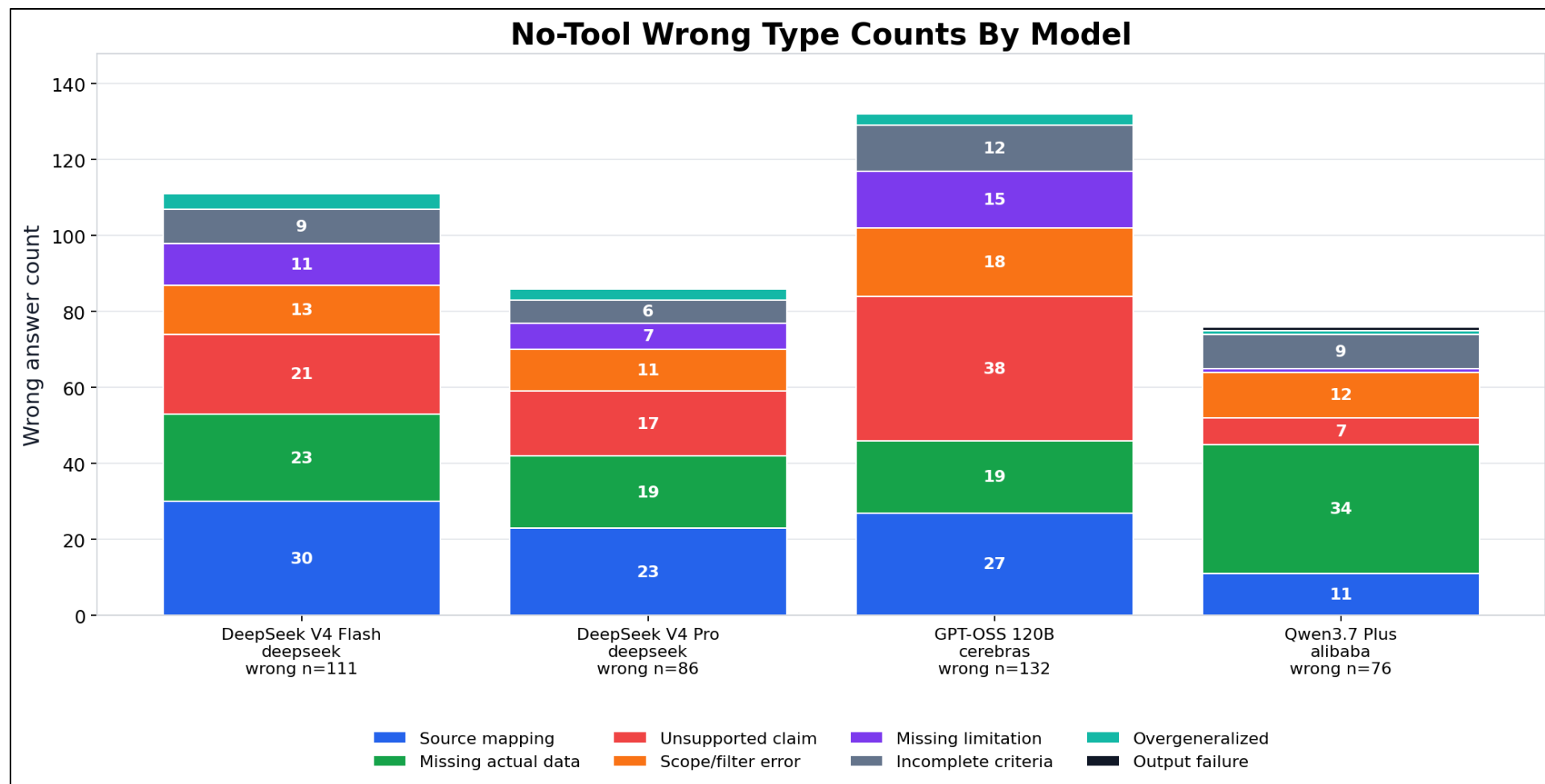
벤치마크 (3)



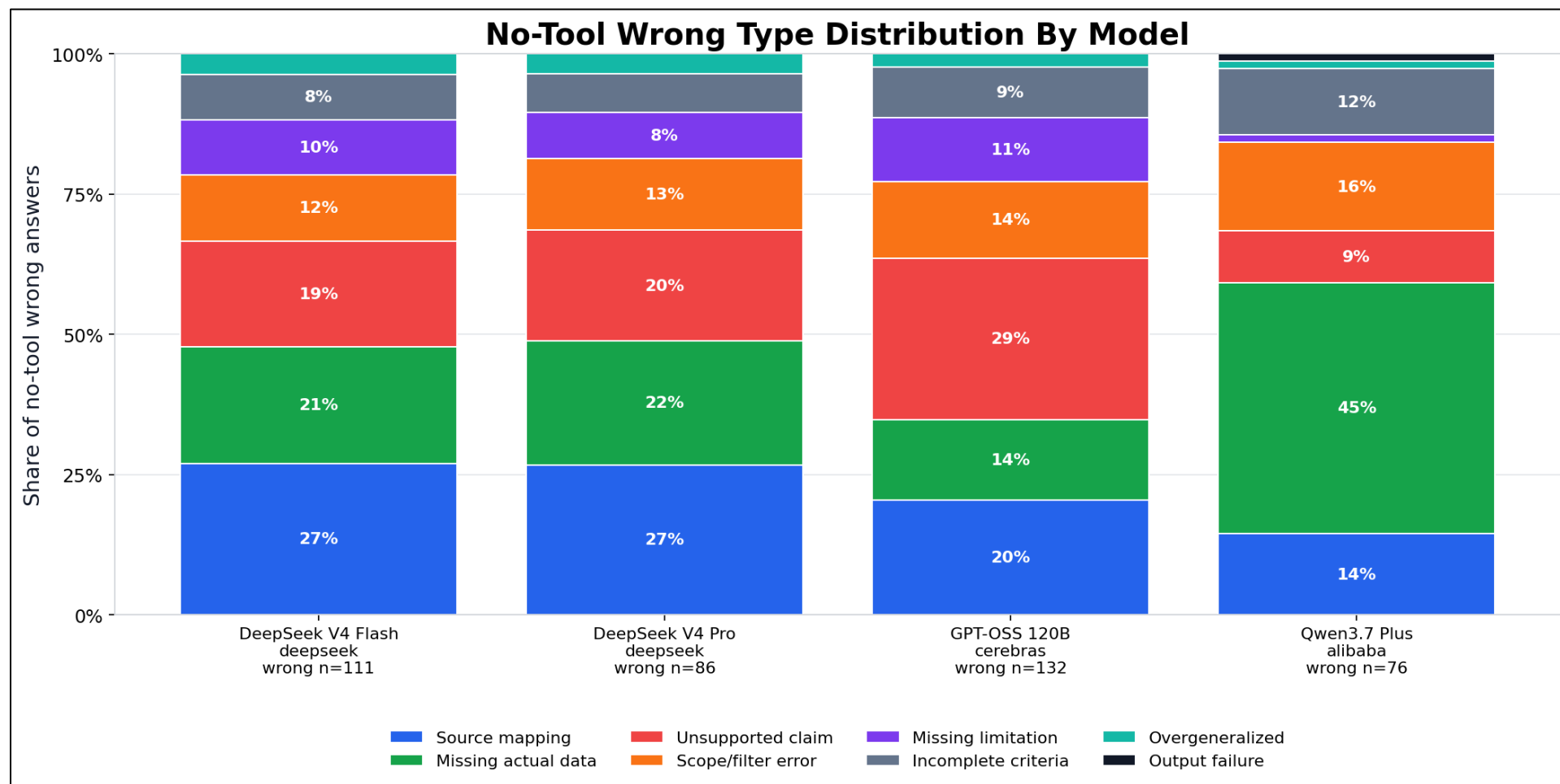
벤치마크 (4)



벤치마크 (5)



벤치마크 (6)





SECTION 04

프로젝트 데모

Streamlit base

04

데모 링크



APPENDIX

Github 링크



E.O.D